# Protected controller connections

The organizer supervisor retains the full answer map, victim/item data, field previews, map review and replay exports. Participant controllers receive their robot's normal device data and competition messages. They cannot use the public controller endpoint to download organizer assets or open a supervisor browser connection.

## Competition server

Use Docker Engine 28 or newer and Docker Compose. Run these commands from the Erebus repository root, on an organizer-controlled computer:

```
python3 scripts/start_competition.py
```

Only port **1234** is public. Participants use the ordinary R2025a launcher, targeting the organizer computer and the participant robot name:

```
webots-controller --protocol=tcp --ip-address=SERVER_IP --port=1234 --robot-name=Erebus_Bot \
robot_controller.py
```

On macOS the executable is inside `Webots.app/Contents/MacOS`. Include `--robot-name=Erebus_Bot`; the gateway checks this exact controller handshake. The gateway checks native request framing (64 MiB per controller request) and rejects browser-protocol switching. Sensor data, motors, emitter/receiver communication, token reporting, map submission/evaluation, game-info/battery replies, relocation and exit use the existing Webots protocol unchanged.

Open the organizer's Match Studio on the server computer:




The private 3D field viewer is also available to the organizer. Webots starts its controller event loop immediately; the match itself waits for Run.

Webots runs on a separate organizer Docker network with outbound access for official model assets. Its browser/assets port is published only on the organizer's loopback address, as is the upload service on 60520. Participant containers join a different network. Do not change these private bindings to `0.0.0.0`, attach participant containers to the organizer network, publish the private Webots port separately, or give participants Docker-daemon access.

Select the robot JSON and start the match from the organizer interface. The isolated server always uses an external controller, cannot disable that separation, and cannot build participant containers inside the organizer process. Start participant programs on their own computer or in the separate participant service.

Use `python3 scripts/start_competition.py --world WORLD_NAME` to select a bundled/private world filename stem. The default is `mapping_example_1_cyan_tiles`. Do not give private world files or the organizer image to participants. Publicly distributed practice maps cannot be made unknown by access controls.

Battery settings remain organizer-configurable in `game/controllers/MainSupervisor/sensorEnergyConfig.txt`. For example, `ChangeBatteryTime = 5` or `10` selects the stationary replacement duration. After editing the source configuration, rerun the protected server launcher to rebuild/recreate the server with those settings.

Results remain organizer-owned. Retrieve them, including full map reviews and replays, with:

```
docker compose -f docker/competition/compose.yaml cp webots:/erebus/match-results \
./organizer-results
```

Open copied map reviews and replays on the organizer computer; the headless server cannot open that computer's browser. Remote organizers should tunnel the loopback ports rather than publish them.

## Participants running on the organizer computer

For a local Docker controller, put the approved program in `docker/controller/robot_controller.py` and use:

```
docker compose -f docker/competition/compose.yaml --profile participant run --build --rm \
participant
```

That container has no Erebus files, result volume, organizer-network membership, privileged capabilities or Docker socket. Its robot connection still goes through the controller gateway. Only the organizer controls the container configuration.

On a Mac with the native R2025a SDK, a restricted local launcher is also provided:

```
python3 scripts/run_restricted_controller.py /path/to/controller/robot_controller.py
```

This copies the dedicated controller directory into temporary storage and uses the Mac OS sandbox. The controller may read its own code and required SDK/system libraries, write its own temporary output and contact the loopback controller gateway. Direct organizer-file reads and connections to other ports are denied. Linked dependencies are rejected. Use a separate controller directory; do not supply the Erebus project as the controller folder.

For a native Webots development instance, the same gateway can be started separately, with the actual private Webots port:

```
python3 scripts/controller_gateway.py --port 1234 --backend-port 1255
```

Enable Remote in that native instance and start the restricted Mac launcher. This protects that local controller process; it does not hide the native backend from other network users. Network competitions must use the isolated server deployment or an equivalently enforced private backend.

**Ordinary unrestricted native Run is development mode, not isolation.** Running participant code directly under the organizer account allows that code the account's file permissions. The restricted launcher and container deployment are explicit protected execution paths; do not claim the unrestricted path is protected.

## Verification and boundaries

Qualification on 3 November 2026: 33 native R2025a cases passed; a separate latest-framing test passed 29 token reports plus map/exit scoring; the final isolated-server run passed organizer-preview access, participant-network denial, denied Supervisor API access and real motion/map/game-info/battery/exit exchanges. All 20 portable regression scripts and the interface contract passed in six OS/Python jobs. These are separate qualification runs, not one combined run. Full deployment testing used Linux; this Mac separately passed the OS file/network denial checks and native SDK startup inside the restricted launcher.

Portable checks exercise HTTP/WebSocket rejection, supervisor-name rejection, fragmented handshakes and byte-for-byte forwarding of a 1 MiB payload. Mac tests exercise denied organizer-file reads, denied private-backend connections and permitted robot traffic/output. Native simulator tests exercise actual sensors, motor motion, token reports, map submission/evaluation and exit through the gateway. The separate container integration test checks that organizer previews remain available while participant-network access fails.

These controls address the tested file/network information paths. They do not certify against unknown Webots, operating-system or container-engine vulnerabilities. Organizer/admin access remains trusted. Replacing the private network with an exposed raw Webots port invalidates the protection.

Networking baseline: Docker loopback publishing and the pre-28 limitation.